

Memoria-Técnica Proyecto Final



Autor: Sergio Estudillo Marabot

Profesor: Fernando Macías Ramos

Módulo: Desarrollo de Interfaces

Centro educativo: IES Rafael Alberti (Cádiz)

Índice

1. Introducción y Contexto.....	4
1.1. Antecedentes y Justificación de la Necesidad.....	4
1.2. Descripción de la Propuesta.....	4
1.3. Objetivos del Proyecto.....	5
1.3.1. Objetivo General.....	5
1.3.2. Objetivos Específicos.....	5
1.4. Público Objetivo.....	5
1.5. Ámbito del Proyecto.....	6
2. Análisis Tecnológico y Herramientas (RA1.a, RA7.d).....	6
2.1. Lenguaje de Programación: Kotlin.....	6
2.2. Entorno de Desarrollo (IDE).....	7
2.3. Framework de Interfaz: Jetpack Compose.....	7
2.4. Arquitectura del Software.....	7
2.5. Gestión de Datos y Persistencia.....	8
2.6. Librerías Externas y APIs del Sistema (RA7.d).....	8
2.7. Control de Versiones.....	9
3. Diseño de Interfaz y Experiencia de Usuario (UI/UX) (RA1, RA2, RA4)	9
3.1. Identidad Visual: "Cyberpunk Ecológico".....	9
3.2. Diseño de Componentes y Jerarquía (RA1.c, RA1.d).....	10
3.3. Interfaz Natural de Usuario (NUI) - (RA2).....	11
3.3.1. Interacción por Voz (RA2.c).....	11
3.3.2. Visión Artificial y Realidad Aumentada (RA2.f).....	11
3.3.3. Biometría (RA2.e).....	11
3.3.4. Gestos (RA2.d).....	12
3.4. Usabilidad y Accesibilidad (RA4.i).....	12
4. Arquitectura y Estructura de Datos (RA3, RA6.d).....	12
4.1. Patrón de Diseño: MVVM + Clean Architecture.....	12
4.2. Estructura de la Información Persistente (Room Database) (RA6.d)..	13
4.3. Componentes Reutilizables (RA3).....	15
4.3.1. DonationCard (Tarjeta Polimórfica).....	15
4.3.2. NeonBorderBox (Contenedor Visual).....	15
4.3.3. InfoDialog (Ayuda Contextual).....	15
4.4. Flujo de Datos y Eventos (RA1.g).....	16
4.5. Análisis Detallado de Implementación (Código Clave) - (RA1.e)....	16
4.6. Sistema de Navegación.....	22
4.7 Estructura de Vistas Principales.....	24
A. LoginScreen (Autenticación).....	24

B. RegisterScreen (Registro de Usuarios).....	26
C. AdminHistoryScreen (Historial pedidos Admin).....	27
D. UserHistoryScreen (Historial pedidos Usuario).....	28
5. Plan de Pruebas y Aseguramiento de la Calidad (QA) (RA8, RA4.i).	29
5.1. Estrategia de Pruebas (RA8.a).....	29
5.2. Pruebas de Integración (RA8.b).....	29
Caso de Prueba 1: Ciclo de Vida de la Donación.....	29
Caso de Prueba 2: Persistencia y Reinicio.....	30
Caso de Prueba 3: Cancelación de Usuario.....	30
5.3. Pruebas de Regresión (RA8.c).....	30
5.4. Pruebas de Usabilidad y Accesibilidad (RA4.i).....	31
5.5. Pruebas de Seguridad (RA8.e).....	31
5.6. Pruebas de Rendimiento, Volumen y Recursos.....	31
5.7. Automatización de Pruebas Unitarias (JUnit & Mockito).....	32
6. Manuales técnico de instalación, Ayudas y Documentación (RA6).	34
6.1. Sistema de Ayudas Integrado (Ayuda Contextual) - (RA6.a, RA6.c)...	34
6.2. Manual Técnico de Instalación y Despliegue - (RA6.f, RA7).....	35
6.2.1. Requisitos del Sistema.....	35
6.2.2. Instalación del Proyecto.....	35
6.2.3. Estructura del Código Principal.....	35
6.2.4. Generación del Ejecutable (APK Firmado) - (RA7.a, RA7.e).....	36
7. Distribución, Instalación y Despliegue (RA7).....	36
7.1. Empaquetado de la Aplicación (RA7.a, RA7.c).....	36
7.2. Firma Digital y Seguridad (RA7.e).....	37
7.3. Personalización del Instalador (RA7.b).....	37
7.4. Estrategia de Distribución (RA7.h).....	38
7.5. Gestión de Instalación y Desinstalación (RA7.f, RA7.g).....	38
8. Conclusiones y escala de posibles mejoras futuras.....	39
8.1. Conclusiones Generales.....	39
8.2. Logros Técnicos Destacados.....	39
8.3. Líneas de Trabajo Futuras (Mejoras).....	40
9. Bibliografía y Referencias.....	40
ANEXO I: Videodemostración del Proyecto.....	42

1. Introducción y Contexto

1.1. Antecedentes y Justificación de la Necesidad

En la actualidad, el desperdicio alimentario representa uno de los mayores desafíos éticos y medioambientales de nuestra sociedad. Según datos de la ONU, un porcentaje significativo de la producción mundial de alimentos se pierde o desperdicia, mientras que una parte de la población sufre inseguridad alimentaria.

A nivel local, pequeños comercios (panaderías, fruterías, tiendas de barrio) a menudo se ven obligados a desechar productos perecederos al final del día que, aunque no son aptos para la venta al día siguiente, están en perfectas condiciones para el consumo inmediato.

FoodShare nace para solucionar este problema de "última milla", actuando como un puente digital ágil entre el excedente comercial y la necesidad social. El proyecto se alinea directamente con los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030, específicamente con el ODS 12 (Producción y Consumo Responsables) y el ODS 2 (Hambre Cero).

1.2. Descripción de la Propuesta

FoodShare es una aplicación móvil nativa desarrollada para el sistema operativo Android, diseñada para optimizar la gestión y recolección de excedentes alimentarios en tiempo real.

A diferencia de otras soluciones logísticas tradicionales, FoodShare apuesta por una experiencia de usuario (UX) inmersiva y moderna, bajo una estética "Cyberpunk Ecológico" que busca atraer tanto a comercios jóvenes como a voluntarios digitales. La aplicación no solo gestiona datos, sino que facilita la interacción humana mediante tecnologías de Interfaz Natural (NUI), eliminando barreras de entrada mediante el uso de voz, biometría y realidad aumentada.

La solución permite:

- A los Comercios (Admin): Publicar ofertas en segundos mediante dictado por voz y gestionar el stock visualmente.
- A los Voluntarios (User): Reservar alimentos y validar su recogida mediante un sistema seguro de códigos QR dinámicos.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Diseñar y desarrollar una solución tecnológica integral que reduzca el desperdicio de alimentos en el comercio local, facilitando su redistribución inmediata mediante una interfaz accesible, segura y altamente eficiente.

1.3.2. Objetivos Específicos

Para alcanzar la meta principal, se han establecido los siguientes objetivos técnicos y funcionales, alineados con los criterios de evaluación del ciclo formativo:

1. Implementar una Arquitectura Robusta: Utilizar el patrón MVVM (Model-View-ViewModel) junto con Clean Architecture para garantizar un código mantenible, escalable y testable (RA1, RA3).
2. Garantizar la Accesibilidad Universal: Integrar herramientas de NUI (Natural User Interface) como reconocimiento de voz y autenticación biométrica para reducir la fricción en el uso de la app (RA2).
3. Asegurar la Integridad de Datos: Implementar un sistema de persistencia local mediante Room Database que soporte el funcionamiento offline-first y mantenga un historial fiable de transacciones (RA6).
4. Optimizar la Logística de Entrega: Desarrollar un sistema de validación presencial mediante escaneo de Códigos QR (Visión Artificial) para garantizar que la entrega se realiza a la persona correcta (RA2).
5. Proveer Análisis de Impacto: Dotar a la aplicación de herramientas de visualización de datos (Gráficos animados) y generación de Informes PDF para que los usuarios sean conscientes de su impacto positivo (RA5).

1.4. Público Objetivo

La aplicación está diseñada para satisfacer las necesidades de dos perfiles de usuario claramente diferenciados:

1. **El Administrador (Comercio Local):** Dueños o empleados de pequeños establecimientos de alimentación. Requieren una interfaz

rápida, que no interrumpa su flujo de trabajo y que permita publicar productos con el mínimo esfuerzo (uso de voz y fotos rápidas).

2. **El Usuario (Voluntario/Beneficiario):** Personas concienciadas con el medio ambiente o con necesidad de acceso a alimentos. Buscan una interfaz clara, visualmente atractiva y que les proporcione seguridad y transparencia en el proceso de reserva y recogida.

1.5. *Ámbito del Proyecto*

Este proyecto se enmarca dentro de los ámbitos de Medio Ambiente (economía circular y reducción de residuos) y Accesibilidad e Inclusión (diseño de interfaces adaptadas y uso de tecnologías asistivas), cumpliendo con los requisitos planteados en el enunciado del Proyecto Final.

2. Análisis Tecnológico y Herramientas (RA1.a, RA7.d)

Para el desarrollo de FoodShare, se ha realizado un análisis exhaustivo de las tecnologías disponibles en el ecosistema Android actual. La selección de herramientas se ha basado en criterios de estabilidad, escalabilidad, mantenibilidad y capacidad para soportar interfaces modernas y reactivas.

A continuación, se detallan y justifican las decisiones tecnológicas adoptadas.

2.1. *Lenguaje de Programación: Kotlin*

Se ha elegido Kotlin como lenguaje único para el desarrollo del proyecto (100% del código base).

- Justificación (RA1.a): Kotlin es el lenguaje oficial recomendado por Google para el desarrollo Android. Su principal ventaja frente a Java es la Null Safety (seguridad contra nulos), que elimina gran parte de los errores en tiempo de ejecución (NullPointerException). Además, su sintaxis concisa y el soporte nativo para Corrutinas permiten gestionar la asincronía (llamadas a base de datos, sensores) de forma eficiente y legible, evitando el "callback hell".

2.2. Entorno de Desarrollo (IDE)

El proyecto ha sido desarrollado utilizando Android Studio (versión Ladybug/Koala), el entorno de desarrollo integrado oficial.

- Herramientas utilizadas:
 - Gradle (Kotlin DSL): Para la gestión de dependencias y la automatización de la construcción del proyecto (Build System).
 - Device Manager: Para la emulación de dispositivos Pixel con diferentes API levels (pruebas de compatibilidad).
 - Database Inspector: Para la depuración en tiempo real de la base de datos Room.
 - Logcat & Profiler: Para el monitoreo de rendimiento (uso de CPU/GPU en animaciones) y depuración de errores.

2.3. Framework de Interfaz: Jetpack Compose

Para la capa de presentación (UI), se ha descartado el sistema tradicional de XML en favor de Jetpack Compose.

- Justificación de Diseño (RA1.b): Compose es el toolkit moderno declarativo de Android. Su uso ha sido fundamental para implementar la estética "Cyberpunk" y las animaciones complejas del proyecto.
 - Permite crear componentes reutilizables (DonationCard, NeonBorderBox) con menos código.
 - Facilita la gestión de estados (State Hoisting), crucial para que la interfaz reaccione a los cambios de datos en tiempo real.
 - Ofrece acceso directo a las APIs de gráficos (Canvas, GraphicsLayer), permitiendo desarrollar el motor de partículas (FloatingFoodBackground) con un alto rendimiento al utilizar aceleración por GPU.

2.4. Arquitectura del Software

El proyecto sigue una arquitectura limpia basada en el patrón MVVM (Model-View-ViewModel) recomendado por Google.

- View (Vista): Pantallas en Compose (AdminHomeScreen, UserHomeScreen). Son "tontas" y solo muestran datos.

- ViewModel: Gestiona la lógica de presentación y el estado de la UI (AdminViewModel). Sobrevive a los cambios de configuración (rotación de pantalla).
- Model (Repository): Abstrae la fuente de datos (EcoRepository). La UI no sabe si los datos vienen de una API o de una BBDD local.

Librería de Inyección de Dependencias: Dagger Hilt Se utiliza Hilt para gestionar la inyección de dependencias. Esto desacopla las clases, facilita el testing y gestiona automáticamente el ciclo de vida de los componentes (Singletons de base de datos, Contextos, etc.).

2.5. Gestión de Datos y Persistencia

Para el almacenamiento local de la información (RA6.d), se utiliza Room Database.

- Justificación: Room proporciona una capa de abstracción sobre SQLite. Permite la verificación de consultas SQL en tiempo de compilación y se integra nativamente con Kotlin Flow, lo que permite que la interfaz se actualice automáticamente ("Reactividad") cuando cambian los datos en la base de datos, sin necesidad de refrescar la pantalla manualmente.

2.6. Librerías Externas y APIs del Sistema (RA7.d)

Para potenciar las funcionalidades avanzadas (NUI y Logística), se han integrado las siguientes bibliotecas:

Librería / API	Función en FoodShare	Justificación
ZXing Android Embedded	Escáner QR	Librería robusta y ligera para la lectura de códigos de barras y QR, esencial para el módulo de validación AR.
Coil (Coroutines Image Loading)	Carga de Imágenes	Carga asíncrona de imágenes optimizada para Compose. Gestiona caché y redimensionado automático.
Biometric API (AndroidX)	Seguridad	Permite el uso de hardware biométrico (huella/rostro) para un login seguro sin gestión compleja de criptografía.
Speech Recognizer	Voz (NUI)	API nativa de Android para convertir voz a texto, utilizada en el dictado de descripciones de productos.
PDF Document API	Informes	API nativa para dibujar documentos vectoriales y exportarlos, utilizada en el módulo de reportes.

Del proyecto destaco el uso de la librería **ZXing Android Embedded**. Ha sido implementada en el diálogo de validación del Administrador para la lectura automática del código QR del voluntario, cerrando el ciclo de entrega.

2.7. Control de Versiones

Se ha utilizado Git para el control de versiones, alojando el código en un repositorio remoto (GitHub). Se ha seguido una estrategia de commits atómicos y descriptivos (Conventional Commits) para documentar la evolución del desarrollo, desde la estructura inicial hasta la implementación de mejoras visuales y funcionales.

3. Diseño de Interfaz y Experiencia de Usuario (UI/UX) (RA1, RA2, RA4)

El diseño de FoodShare se aleja de los estándares corporativos tradicionales para ofrecer una experiencia inmersiva y gamificada. Se ha adoptado una filosofía de diseño Dark Mode First (Modo Oscuro Prioritario) combinada con elementos de alto contraste, garantizando tanto la estética como la accesibilidad.

3.1. Identidad Visual: "Cyberpunk Ecológico"

La línea gráfica del proyecto fusiona la tecnología avanzada con la sostenibilidad.

- **Paleta de Colores (RA4.g)**

- **Fondo: Negro Puro (#0D0D0D) y Gris Antracita (#1E1E1E).** Esta elección reduce la fatiga visual en entornos de baja luz y minimiza el consumo de batería en pantallas OLED/AMOLED.
- **Acento Principal: Verde Neón (#00FF41).** Utilizado en acciones primarias (Login, Publicar) y en el sistema de partículas. Este color ofrece un ratio de contraste superior a 15:1 sobre fondo negro, superando ampliamente el estándar WCAG AAA de accesibilidad.
- **Acento Secundario: Naranja Alerta (#FFAB40) y Azul Cian (#00E5FF).** Utilizados para estados de advertencia (Reservado) y éxito (Completado).

- **Motor de Partículas (Floating Food):** Se ha desarrollado un componente visual personalizado (**VisualEffects.kt**) que renderiza iconos de alimentos flotantes en el fondo.
 - *Justificación Técnica:* Para no comprometer el rendimiento, la animación utiliza **Modifier.graphicsLayer**, delegando el renderizado a la **GPU** del dispositivo en lugar de la CPU, garantizando una fluidez constante de 60 FPS (RA1.f).

3.2. Diseño de Componentes y Jerarquía (RA1.c, RA1.d)

La interfaz sigue las directrices de **Material Design 3**, pero con una personalización profunda de los componentes para adaptarlos a la identidad de marca.

- **Tarjetas Interactivas (DonationCard):** Se ha diseñado una tarjeta modular que adapta su contenido según el rol (Admin ve botones de borrar/validar; Usuario ve botón de reservar). Utiliza elevación y sombras de color (**spotShadowColor**) para crear profundidad.
- **Feedback Visual (NeonBorderBox):** Para destacar las acciones más importantes (CTA - Call to Action), se ha creado un contenedor con bordes de degradado animado. Esto guía la atención del usuario de forma subconsciente hacia el botón principal de cada pantalla.
- **Navegación:** Se utiliza una estructura de **Scaffold** con una **TopAppBar** limpia que agrupa las acciones secundarias (Ayuda, Historial, Salir) en un menú desplegable (**DropDownMenu**), priorizando el contenido principal (RA4.c).

3.3. Interfaz Natural de Usuario (NUI) - (RA2)

Uno de los pilares de FoodShare es reducir la fricción mediante interacciones naturales, cumpliendo con los criterios avanzados de la rúbrica.

3.3.1. Interacción por Voz (RA2.c)

En el panel de administración, se ha integrado la API de reconocimiento de voz de Android.

- **Implementación:** Un botón de micrófono en el campo "Descripción" permite al comerciante dictar los detalles del producto en lugar de escribirlos.
- **Beneficio:** Agiliza drásticamente el proceso de publicación, permitiendo al comerciante subir ofertas mientras manipula los productos con las manos.

3.3.2. Visión Artificial y Realidad Aumentada (RA2.f)

Se utiliza la cámara del dispositivo como un sensor inteligente para la logística de entrega.

- **Implementación:** Se implementa un sistema de **Realidad Aumentada basada en marcadores (Marker-based AR)**
- **Flujo:** A diferencia de la AR posicional (ARCore), este sistema utiliza el patrón físico del QR como 'anchor' (ancla) para superponer información digital (la validación y el nombre del usuario) directamente sobre la visión del mundo real del dispositivo.

3.3.3. Biometría (RA2.e)

Para el inicio de sesión, se ha implementado la autenticación biométrica (Huella Dactilar / Reconocimiento Facial) mediante **BiometricPrompt**.

- **Justificación:** Aumenta la seguridad al no requerir teclear contraseñas en público y reduce el tiempo de acceso a la aplicación a menos de 2 segundos.

3.3.4. Gestos (RA2.d)

Se ha implementado el patrón "**Swipe to Delete**" (Deslizar para borrar) en las listas del administrador.

- **Interacción:** El usuario puede deslizar una tarjeta hacia la izquierda o derecha para eliminar una oferta, una interacción táctil intuitiva y estándar en aplicaciones móviles modernas.

3.4. Usabilidad y Accesibilidad (RA4.i)

El diseño ha sido sometido a pruebas heurísticas para garantizar su facilidad de uso:

1. **Visibilidad del estado del sistema:** El usuario siempre sabe qué está pasando gracias a los mensajes **Toast** ("Publicado correctamente", "Huella reconocida") y a los cambios de estado visuales en las tarjetas (etiquetas "DISPONIBLE" vs "RESERVADO").
2. **Prevención de errores:** Los campos de texto tienen validación en tiempo real (no se puede publicar si falta el título). Los botones críticos, como el borrado, requieren confirmación o gestos deliberados.
3. **Consistencia:** Todos los botones de acción principal comparten el mismo estilo (Verde Neón), y todos los botones secundarios o de salida mantienen un estilo de borde o color de alerta (Naranja).

4. Arquitectura y Estructura de Datos (RA3, RA6.d)

La robustez de **FoodShare** reside en su arquitectura de software. Se ha implementado una solución escalable y desacoplada siguiendo los principios de **Clean Architecture** y el patrón de diseño **MVVM (Model-View-ViewModel)**, estándar recomendado por Google para el desarrollo moderno en Android.

4.1. Patrón de Diseño: MVVM + Clean Architecture

La aplicación se divide en tres capas claramente diferenciadas, lo que facilita el testeo y el mantenimiento del código (RA1.e).

1. **Capa de Presentación (UI):**
 - Implementada con **Jetpack Compose**.
 - **Responsabilidad:** Mostrar los datos al usuario y capturar eventos (clicks, gestos).

- **Componentes:** AdminHomeScreen, UserHomeScreen, LoginScreen.
 - **Comunicación:** Observa el *StateFlow* del ViewModel y se redibuja automáticamente cuando el estado cambia.
2. **Capa de Dominio/Lógica (ViewModel):**
- **Responsabilidad:** Actuar como intermediario entre la UI y los datos. Contiene la lógica de negocio (validaciones, cálculos de stock).
 - **Componentes:** AdminViewModel, UserViewModel.
 - **Inyección:** Recibe los repositorios mediante **Dagger Hilt**, asegurando un bajo acoplamiento.
3. **Capa de Datos (Data Layer):**
- **Responsabilidad:** Fuente única de verdad (*Single Source of Truth*). Decide si los datos vienen de la base de datos local o (en una futura expansión) de una API remota.
 - **Componentes:** EcoRepository, AppDatabase (Room).

4.2. Estructura de la Información Persistente (Room Database) (RA6.d)

La persistencia de datos es crucial para permitir el funcionamiento offline. Se gestiona mediante **Room**, una librería de abstracción sobre SQLite que garantiza la integridad y fluidez en el acceso a datos.

Entidad Principal: DonationEntity La tabla central del sistema es donations, diseñada para soportar el ciclo de vida completo del producto sin borrado físico (Soft Delete).

- Validación de Email (UNICIDAD)

Antes de crear un nuevo usuario, el **ViewModel** realiza una consulta previa a la base de datos para comprobar la unicidad del correo electrónico. Si el email ya existe, se aborta el registro y se notifica al usuario mediante un mensaje Toast, garantizando la integridad de los datos.

Campo	Tipo	Descripción	Restricciones
id	Int	Identificador único	Primary Key
title	String	Nombre del producto	Not null
description	String	detalles o dictado por voz	-
quantity	String	Cantidad disponible	Not null
imageUrl	String	URL de la imagen del producto	-
pickupCode	String	PIN de seguridad generado	Unique (lógica de negocio)
isReserved	Boolean	Estado de reserva	Default: false
isCompleted	Boolean	Estado de entrega	Default: false
isCanceled	Boolean	Indica si el pedido fue cancelado (Soft Delete). Permite mantener el historial.	-
userPhone	String?	Teléfono del voluntario que reservó	-
contactPhone	String	Teléfono del comercio	-

Diagrama de Relación: Dado que es una arquitectura simplificada para demostración, se utiliza una tabla única que gestiona estados lógicos mediante *flags* booleanos (**isReserved**, **isCompleted**), permitiendo consultas rápidas y filtrado eficiente mediante flujos reactivos (**Flow**) en los ViewModels.

4.3. Componentes Reutilizables (RA3)

Para garantizar la consistencia visual y reducir la duplicidad de código, se han creado componentes propios altamente parametrizables (RA3.b, RA3.c).

4.3.1. DonationCard (Tarjeta Polimórfica)

Este componente es el núcleo de las listas de productos. Gestiona la comunicación bidireccional. Incluye lógica de **Prioridad de Estados**: Si el pedido está 'cancelado', la tarjeta se renderiza en rojo y bloquea las acciones de reserva, independientemente de otros estados. Además, integra **Intent** implícitos para realizar llamadas telefónicas entre Comercio y Voluntario directamente desde la UI. Es capaz de adaptar su apariencia y comportamiento según el contexto:

Lógica Interna: El componente utiliza un **Card** de **Material3** con un modificador de borde personalizado. La imagen del producto se carga de forma asíncrona mediante **Coil (AsyncImage)**, gestionando automáticamente la caché y el redimensionado para evitar bloqueos en el hilo principal (Main Thread).

- **Modo Admin:** Muestra botones de "Borrar" y permite la validación mediante click para abrir el diálogo de escáner.
- **Modo Usuario:** Muestra el botón de "Reservar" si está disponible, o el Código QR si ya ha sido reservado.
- **Parámetros:** Recibe lambdas (**onActionClick: () -> Unit**) para delegar la lógica al componente padre (Hoisting), cumpliendo con el principio de responsabilidad única (RA3.d).

4.3.2. NeonBorderBox (Contenedor Visual)

Un componente contenedor ("Wrapper") que aplica un borde con degradado animado (**Brush.sweepGradient**) alrededor de cualquier contenido que se le pase.

- **Uso:** Se utiliza para envolver los botones de acción principal ("Iniciar Sesión", "Publicar"), atrayendo la atención del usuario mediante animación visual sutil.

4.3.3. InfoDialog (Ayuda Contextual)

Un componente de diálogo genérico utilizado para mostrar instrucciones y ayuda en cada pantalla (RA6.c).

- **Reutilización:** Se instancia en todas las pantallas (**AdminHomeScreen**, **UserHomeScreen**) pasando simplemente el título y el texto descriptivo, estandarizando la forma en que se presenta la ayuda al usuario.

4.4. Flujo de Datos y Eventos (RA1.g)

La aplicación sigue un flujo de datos unidireccional (*Unidirectional Data Flow*):

1. **Evento:** El usuario realiza una acción (ej. pulsar "Reservar").
2. **ViewModel:** La UI notifica al ViewModel llamando a una función (**viewModel.reserveDonation()**).
3. **Estado:** El ViewModel actualiza la base de datos a través del Repositorio.
4. **Observación:** La base de datos emite el nuevo estado a través de un **Flow**.
5. **Renderizado:** La UI observa este flujo y se actualiza automáticamente mostrando el nuevo estado (ej. la tarjeta pasa a "Reservado").

4.5. Análisis Detallado de Implementación (Código Clave) - (RA1.e)

Para demostrar el dominio sobre el código desarrollado, a continuación se analizan tres fragmentos críticos que definen la funcionalidad avanzada de la aplicación.

A. Lógica Reactiva en el ViewModel (MVVM) → En lugar de utilizar callbacks tradicionales, se ha implementado un flujo reactivo utilizando **StateFlow**. Esto garantiza que la interfaz de usuario (UI) siempre refleje el estado actual de la base de datos, incluso tras cambios asíncronos.

AdminViewModel.kt:

```
3  > import ...
15
16  /**
17   * ViewModel principal para la gestión del comercio (Admin).
18   * Implemento la lógica de negocio para gestionar el ciclo de vida de las donaciones.
19   *
20   * RA3.e: Utilizo clases y métodos para gestionar la lógica de la aplicación.
21   * @property repository Repositorio de datos (Inyectado por Hilt).
22   */
16 Usages
23  @HiltViewModel
24  class AdminViewModel @Inject constructor(
25      private val repository: EcoRepository
26  ) : ViewModel() {
27
28      3 Usages
29      private val _currentFilter = MutableStateFlow(value = "TODOOS")
30      val currentFilter = _currentFilter.asStateFlow()
31
32      /**
33       * RA5.c: Establezco filtros sobre los valores a presentar.
34       * CORRECCIÓN: Los cancelados NO deben salir ni en "Disponibles" ni en "Reservados" de la lista normal.
35       */
36      1 Usage
37      val donationList = repository.getActiveDonations().combine(flow = _currentFilter) { list, filter ->
38          // Filtro base: Ocultar cancelados de la lista de gestión diaria
39          val activeList = list.filter { !it.isCancelled }
40
41          when (filter) {
42              "DISPONIBLES" -> activeList.filter { !it.isReserved }
43              "RESERVADOS" -> activeList.filter { it.isReserved }
44              else -> activeList
```

```
93  /**
94   * RA2.f / RA3.d: Implemento la validación de seguridad (PIN/QR).
95   */
2 Usages
96  fun completeDonation(donation: DonationEntity, inputCode: String): Boolean {
97      if (donation.pickupCode == inputCode) {
98          viewModelScope.launch {
99              repository.completeDonation(donation.id)
100          }
101          return true
102      }
103      return false
104  }
```

- **Análisis:** El uso de **viewModelScope** previene fugas de memoria, cancelando la operación si el usuario sale de la pantalla antes de que termine.

B. Generación de Gráficos con Canvas (Custom Painting) Para el gráfico circular de estadísticas (RA5.e), no se utilizó una librería externa, sino que se dibujó directamente sobre el Canvas nativo de Compose para máximo rendimiento y personalización (segunda imagen a continuación).

PieChart.kt:

```
17  /**
18   * RA5.e: Incluyo gráficos generados a partir de los datos.
19   * Componente personalizado con Canvas para visualizar el estado del stock.
20   * He añadido soporte para visualizar "Reservados" además de disponibles y vendidos.
21   */
22  @Composable
23  fun PieChart(
24      available: Int,
25      reserved: Int,
26      completed: Int,
27      modifier: Modifier = Modifier
28  ) {
29      val total = (available + reserved + completed).toFloat()
30
31      // Animación de entrada suave
32      var animationPlayed by remember { mutableStateOf(value = false) }
33      val animateSize by animateFloatAsState(
34          targetValue = if (animationPlayed) 1f else 0f,
35          animationSpec = tween(durationMillis = 1000),
36          label = "chartAnim"
37      )
38
39      LaunchedEffect(key1 = Unit) { animationPlayed = true }
40
41      Canvas(modifier = modifier.size(size = 220.dp)) {
42          val chartSize = size.minDimension
43          val radius = chartSize / 2
44          val strokeWidth = 50.dp.toPx()
45
46          if (total == 0f) {
```

```
41      Canvas(modifier = modifier.size(size = 220.dp)) {
42          val chartSize = size.minDimension
43          val radius = chartSize / 2
44          val strokeWidth = 50.dp.toPx()
45
46          if (total == 0f) {
47              drawCircle(
48                  color = Color.DarkGray.copy(alpha = 0.3f),
49                  radius = radius,
50                  style = Stroke(width = strokeWidth)
51              )
52          } else {
53              val availableAngle = (available / total) * 360f
54              val reservedAngle = (reserved / total) * 360f
55              val completedAngle = (completed / total) * 360f
56
57              var currentAngle = -90f
58
```

Aquí uso Canvas nativo.

```

59 // 1. COMPLETADOS (Azul - Éxito)
60 drawArc(
61     color = Color( color = 0xFF2196F3),
62     startAngle = currentAngle,
63     sweepAngle = completedAngle * animateSize,
64     useCenter = false,
65     style = Stroke(width = strokeWidth),
66     size = Size( width = chartSize, height = chartSize),
67     topLeft = Offset( x = (size.width - chartSize) / 2, y = (size.height - chartSize) / 2)
68 )
69 currentAngle += completedAngle * animateSize
70
71 // 2. RESERVADOS (Naranja - Pendiente)
72 drawArc(
73     color = AcentoNaranja,
74     startAngle = currentAngle,
75     sweepAngle = reservedAngle * animateSize,
76     useCenter = false,
77     style = Stroke(width = strokeWidth),
78     size = Size( width = chartSize, height = chartSize),
79     topLeft = Offset( x = (size.width - chartSize) / 2, y = (size.height - chartSize) / 2)
80 )
81 currentAngle += reservedAngle * animateSize
82
83 // 3. DISPONIBLES (Verde - Stock)
84 drawArc(
85     color = VerdePrincipal,
86     startAngle = currentAngle,
87     sweepAngle = availableAngle * animateSize,
88     useCenter = false,
89     style = Stroke(width = strokeWidth),
90     size = Size( width = chartSize, height = chartSize),
91     topLeft = Offset( x = (size.width - chartSize) / 2, y = (size.height - chartSize) / 2)
92 )
93 }
94 }

```

- **Análisis:** Este enfoque demuestra el control sobre la API gráfica de bajo nivel de Android, permitiendo animaciones fluidas que serían imposibles con componentes estándar.

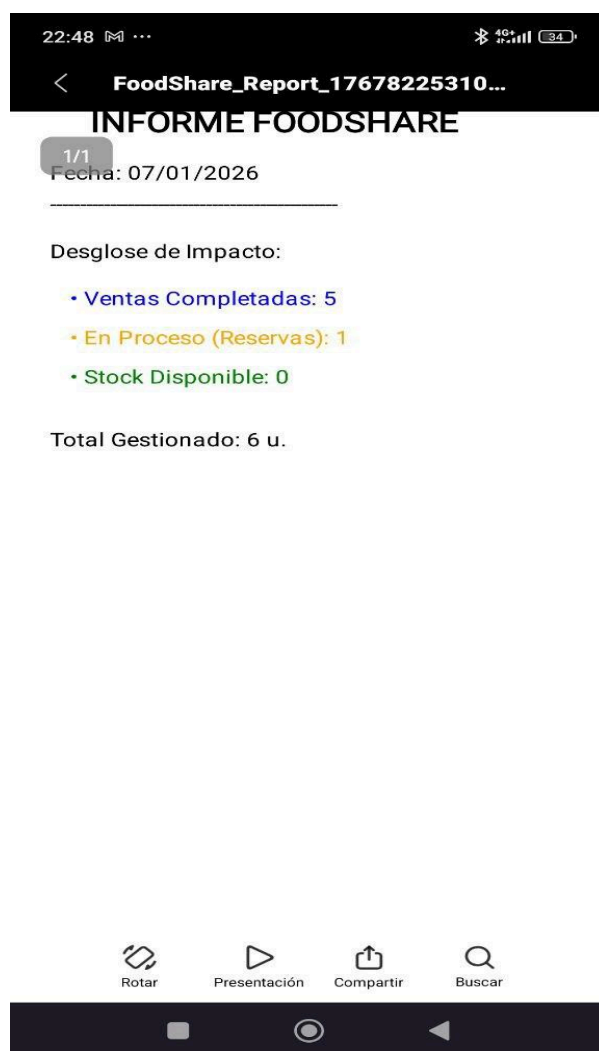
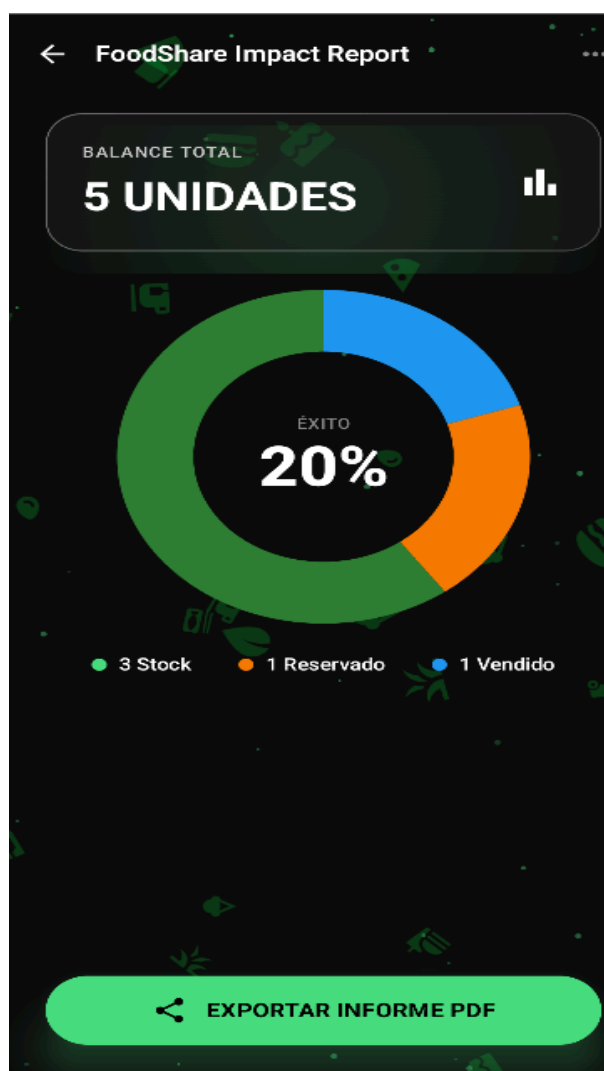
C. Exportación de Informes PDF: La generación del informe no es una captura de pantalla, sino un documento vectorial generado programáticamente utilizando la API **PdfDocument**.

Estrategia Híbrida de Informes: El sistema de informes de FoodShare se ha diseñado bajo un enfoque híbrido para maximizar la utilidad:

1. **Informe Interactivo (En App):** Se prioriza la visualización de datos en tiempo real. Aquí es donde se implementan los **Filtros Dinámicos** (Chips de selección: "Todo", "Alertas", "Completados") y

los **Gráficos Animados** (PieChart). Esto permite al usuario interactuar con la información (RA5.c, RA5.e).

2. **Informe Administrativo (PDF):** Se genera un documento estático vectorial diseñado para la impresión o el archivo burocrático. Este documento condensa los KPI (Indicadores Clave de Desempeño) calculados, como el porcentaje de éxito y el desglose numérico, sirviendo como "acta" de la actividad (RA5.b, RA5.a).



Algunas partes relevantes de [ReportScreen.kt](#):

```
154 // RA5.a: Establezco la estructura del informe PDF
155 fun generatePDF(context: Context, av: Int, res: Int, comp: Int, total: Int) {
156     val pdfDocument = PdfDocument()
157     val pageInfo = PdfDocument.PageInfo.Builder( pageWidth = 300, pageHeight = 600, pageNumber = 1).create()
158     val page = pdfDocument.startPage(pageInfo)
159     val canvas = page.canvas
160     val paint = Paint()
161
162     // Cabecera
163     paint.color = AndroidColor.BLACK
164     paint.textSize = 18f
165     paint.isFakeBoldText = true
166     canvas.drawText( text = "INFORME FOODSHARE", x = 40f, y = 50f, paint)
```

```
173 // Datos detallados
174 canvas.drawText( text = "Desglose de Impacto:", x = 20f, y = 130f, paint)
175
176 paint.color = AndroidColor.BLUE
177 canvas.drawText( text = "• Ventas Completadas: $comp", x = 30f, y = 160f, paint)
```

Bloque de directorio y lógica de almacenamiento

```
190 // Guardado en almacenamiento (RA6.b)
191 val filename = "FoodShare_Report_${System.currentTimeMillis()}.pdf"
192 val contentValues = ContentValues().apply {
193     put(MediaStore.MediaColumns.DISPLAY_NAME, filename)
194     put(MediaStore.MediaColumns.MIME_TYPE, "application/pdf")
195     put(MediaStore.MediaColumns.RELATIVE_PATH, Environment.DIRECTORY_DOWNLOADS)
196 }
197
198 try {
199     val uri = context.contentResolver.insert( uri = MediaStore.Files.getContentUri( volumeName = "external"), contentValues)
200     uri?.let {
201         context.contentResolver.openOutputStream( uri = it)?.use { output ->
202             pdfDocument.writeTo(output)
203         }
204     }
205     Toast.makeText(context, text = "PDF Guardado en Descargas", duration = Toast.LENGTH_LONG).show()
```

4.6. Sistema de Navegación

La navegación entre pantallas se gestiona mediante la librería nativa **Jetpack Navigation Compose**, abandonando el uso de Fragmentos tradicional. Se ha implementado una arquitectura de navegación basada en rutas tipadas para garantizar la seguridad en tiempo de compilación.

- **Gestión de Rutas (Sealed Classes):** Para evitar errores de escritura ("typos") en las cadenas de texto de las rutas, se ha definido una clase sellada Screen que centraliza todos los destinos de la app.

[Screen.kt:](#)

```
3 @_v sealed class Screen(val route: String) {  
4     4 Usages  
5     object Login : Screen(route = "login")  
6     2 Usages  
7     object Register : Screen(route = "register")  
8     3 Usages  
9     object AdminHome : Screen(route = "admin_home") // Pantalla del Comercio  
10    2 Usages  
11    object UserHome : Screen(route = "user_home") // Pantalla del Voluntario  
12    2 Usages  
13    object Report : Screen(route = "report") // pantalla de reportes por PDF  
14    2 Usages  
15    object UserHistory : Screen(route = "user_history_screen")  
16 }
```

Grafo de Navegación (NavHost): El **MainActivity** aloja el **NavHost**, que actúa como el controlador central. Dependiendo del estado de la sesión (gestionado por el ViewModel), el grafo decide si dirigir al usuario a la pantalla de Login o al panel principal correspondiente a su rol.

MainActivity.kt:

```
MainActivity.kt x UserViewModel.kt UserHomeScreen.kt AdminViewModel.kt CurrentUser.kt
29 @AndroidEntryPoint
30 class MainActivity : AppCompatActivity() { // <--- HEREDA DE AppCompatActivity
31     override fun onCreate(savedInstanceState: Bundle?) {
32         super.onCreate(savedInstanceState)
33         setContent {
34             EcoRescueAppTheme {
35                 Surface(
36                     modifier = Modifier.fillMaxSize(),
37                     color = MaterialTheme.colorScheme.background
38                 ) {
39                     AppNavigation()
40                 }
41             }
42         }
43     }
44 }
45
46 1 Usage
47 @Composable
48 fun AppNavigation() {
49     val navController = rememberNavController()
50
51     NavHost(navController = navController, startDestination = Screen.Login.route) {
52         composable(Screen.Login.route) { LoginScreen(navController) }
53         composable(Screen.AdminHome.route) { AdminHomeScreen(navController) }
54         composable(Screen.UserHome.route) { UserHomeScreen(navController) }
55         composable(Screen.Register.route) { RegisterScreen(navController) }
56         composable(Screen.Report.route) { ReportScreen(navController) }
57         composable(Screen.UserHistory.route) { UserHistoryScreen(navController) }
58         composable(Screen.AdminHistory.route) { AdminHistoryScreen(navController) }
59     }
60 }
```

Detalle de Implementación: Módulo Principal (AppModule) El archivo **AppModule.kt** es el componente crítico para la Inyección de Dependencias. Anotado con **@Module** y **@InstallIn(SingletonComponent::class)**, este archivo se encarga de instanciar y proveer las dependencias que deben vivir durante todo el ciclo de vida de la aplicación (Singletons).

Es aquí donde se crea la instancia única de la Base de Datos (**Room**) y se inyecta en el Repositorio, desacoplando la creación de objetos de su uso.

[Appmodule.kt](#):

```
3 > import ...
13
14 @Module
15 @InstallIn(...value = SingletonComponent::class)
16 object AppModule {
17
18     // 1. Enseñamos a Hilt a crear la BASE DE DATOS
19     1 Usage
20     @Provides
21     @Singleton
22     fun provideDatabase(@ApplicationContext context: Context): EcoDatabase {
23         return Room.databaseBuilder(
24             context,
25             klass = EcoDatabase::class.java,
26             name = "foodshare_database" // Le ponemos nombre nuevo para que empiece limpia
27         )
28         .fallbackToDestructiveMigration() // Esto evita crashes al cambiar tablas
29         .build()
30     }
31
32     // 2. Enseñamos a Hilt a obtener el DAO desde la Base de Datos
33     1 Usage
34     @Provides
35     @Singleton
36     fun provideDonationDao(database: EcoDatabase): DonationDao {
37         return database.donationDao()
38     }
39 }
```

4.7 Estructura de Vistas Principales

A. LoginScreen (Autenticación)

Esta pantalla es la puerta de entrada a la aplicación y destaca por integrar múltiples métodos de validación en una sola interfaz limpia.

- **Gestión de Estado:** Utiliza **LaunchedEffect** para comprobar si el dispositivo tiene hardware biométrico disponible al cargar la pantalla.
- **Biometría:** Integra la librería **BiometricPrompt** de **AndroidX**. Si la autenticación es exitosa, el ViewModel recibe un token de validación y navega automáticamente al panel correspondiente (Admin o User) sin necesidad de introducir credenciales manuales.

LoginScreen.kt:

```
36  /**
37   * Defino la pantalla de inicio de sesión de mi aplicación.
38   * En esta pantalla, gestiono la entrada de credenciales del usuario,
39   * la autenticación biométrica y la navegación a las pantallas principales
40   * después de un inicio de sesión exitoso.
41   *
42   * @param navController El controlador de navegación que utilizo para moverme entre pantallas.
43   * @param viewModel El ViewModel que contiene la lógica de negocio para el inicio de sesión.
44   */
45   2 Usages
46   @Composable
47   fun LoginScreen(
48       navController: NavController,
49       viewModel: LoginViewModel = hiltViewModel()
50   ) {
51       /**
52        * Declaro los estados para el correo electrónico, la contraseña y la disponibilidad
53        * de la autenticación biométrica. Utilizo `remember` y `mutableStateOf`
54        * para que la UI se recomponga automáticamente cuando estos valores cambien.
55        */
56       var email by remember { mutableStateOf(value = "") }
57       var password by remember { mutableStateOf(value = "") }
58       var canUseBiometric by remember { mutableStateOf(value = false) }
59       val context = LocalContext.current
```

```
76   FloatingFoodBackground()
77
78   /**
79    * Centro el contenido de la pantalla de inicio de sesión, tanto vertical
80    * como horizontalmente, utilizando una `Column`.
81    */
82   Column(
83       modifier = Modifier
84           .fillMaxSize()
85           .padding(all = 24.dp),
86       horizontalAlignment = Alignment.CenterHorizontally,
87       verticalArrangement = Arrangement.Center
88   ) {
89       /**
90        * Muestro el logo de mi aplicación. He diseñado un `Box` circular con
91        * una sombra de color neón y un borde para que destaque.
92        */
93       Box(
94           contentAlignment = Alignment.Center,
95           modifier = Modifier
96               .size(size = 120.dp)
97               .shadow(elevation = 20.dp, spotColor = VerdePrincipal, shape = CircleShape)
98               .clip(CircleShape)
99               .background(Color(color = 0xFF1E1E1E))
100               .border(width = 2.dp, color = VerdePrincipal, CircleShape)
101       ) {
```

B. RegisterScreen (Registro de Usuarios)

Pantalla diseñada para la captación de nuevos voluntarios y comercios.

- Validación Reactiva: Los campos de texto (**OutlinedTextField**) actualizan sus variables de estado (name, email, password) en tiempo real.

- Feedback al Usuario: El botón de registro incluye lógica condicional que impide la llamada a la base de datos si los campos están vacíos, mostrando un **Toast** informativo para guiar al usuario.
- Diseño Visual: Implementa el componente **FloatingFoodBackground** en una capa inferior (Box) para mantener la coherencia estética con el resto de la aplicación.

[RegisterScreen.kt:](#)

```

41
42 // Definimos los colores del tema Glass/Neon para esta pantalla
43 val NeonGreenRegister = Color( color = 0xFF4ADE80)
44 val GlassBg = Color( color = 0xFF1E1E1E).copy(alpha = 0.5f)
45
46 @OptIn( ...markerClass = ExperimentalMaterial3Api::class)
47 @Composable
48 fun RegisterScreen(
49     navController: NavController,
50     viewModel: RegisterViewModel = hiltViewModel()
51 ) {
52     // --- ESTADOS (Lógica original intacta) ---
53     var name by rememberSaveable { mutableStateOf( value = "" ) }
54     var email by rememberSaveable { mutableStateOf( value = "" ) }
55     var phone by rememberSaveable { mutableStateOf( value = "" ) }
56     var password by rememberSaveable { mutableStateOf( value = "" ) }
57     var selectedRole by rememberSaveable { mutableStateOf( value = "USER" ) } // "USER" o "ADMIN"
58     val context = LocalContext.current
59
60     Scaffold(
61         containerColor = Color( color = 0xFF000000), // Fondo base oscuro
62         topBar = {
63             TopAppBar(
64                 title = { },
65                 navigationIcon = {
66                     IconButton(onClick = { navController.popBackStack() }) {
67                         Icon( imageVector = Icons.AutoMirrored.Filled.ArrowBack, contentDescription = null, tint = Color.White)
68                     }
69                 },
70                 colors = TopAppBarDefaults.topAppBarColors(containerColor = Color.Transparent)
71             )
72         }

```

```
// --- FORMULARIO (Estilo Glass) ---

// 1. Nombre
GlassRegisterTextField(
    value = name,
    onValueChange = { name = it },
    placeholder = "Nombre Completo",
    icon = Icons.Default.Person
)
Spacer(modifier = Modifier.height( height = 12.dp))

// 2. Email
GlassRegisterTextField(
    value = email,
    onValueChange = { email = it },
    placeholder = "Email",
    icon = Icons.Default.Email,
    keyboardType = KeyboardType.Email
)
Spacer(modifier = Modifier.height( height = 12.dp))

// 3. Teléfono
GlassRegisterTextField(
    value = phone,
    onValueChange = { phone = it },
    placeholder = "Teléfono Móvil",
    icon = Icons.Default.Phone,
```

C. AdminHistoryScreen (Historial pedidos Admin)

pantalla de auditoría para el comercio. Utiliza un filtrado inverso en el **AdminViewModel** para mostrar exclusivamente los pedidos que han salido del ciclo activo (Completados o Cancelados), permitiendo al administrador tener un registro de la actividad pasada y detectar cancelaciones de usuarios.

```
24 @Composable
25 fun AdminHistoryScreen(
26     navController: NavController,
27     viewModel: AdminViewModel = hiltViewModel()
28 ) {
29     val historyList by viewModel.adminHistory.collectAsState(initial = emptyList())
30
31     Scaffold(
32         containerColor = Color( color = 0xFF000000),
33         topBar = {
34             TopAppBar(
35                 title = {
36                     Text(
37                         text = "Historial de Pedidos",
38                         color = Color.White,
39                         fontWeight = FontWeight.SemiBold,
40                         fontSize = 18.sp
41                     )
42                 },
43                 navigationIcon = {
44                     IconButton(onClick = { navController.popBackStack() }) {
45                         Icon( imageVector = Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Volver", tint = Color.White)
46                     }
47                 },
48                 colors = TopAppBarDefaults.topAppBarColors(containerColor = Color.Transparent)
49             )
50         }
51     ) { padding ->
52         Box(modifier = Modifier.fillMaxSize()) {
53             // --- FONDO DOBLE ---
54             FloatingFoodBackground()
55             FloatingParticlesOverlay()
```

```

51     ) { padding ->
52         Box(modifier = Modifier.fillMaxSize()) {
53             // --- FONDO DOBLE ---
54             FloatingFoodBackground()
55             FloatingParticlesOverlay()
56
57             if (historyList.isEmpty()) {
58                 Box(
59                     modifier = Modifier.fillMaxSize().padding( paddingValues = padding),
60                     contentAlignment = Alignment.Center
61                 ) {
62                     Text( text = "No hay historial disponible.", color = Color.Gray)
63                 }
64             } else {
65                 LazyColumn(
66                     modifier = Modifier
67                         .fillMaxSize()
68                         .padding( paddingValues = padding)
69                         .padding(horizontal = 16.dp),
70                     contentPadding = PaddingValues(top = 8.dp, bottom = 24.dp)
71                 ) {
72                     items( items = historyList, key = { it.id }) { donation ->
73                         // USAMOS LA TARJETA CON IMAGEN
74                         // No pasamos onCallClick porque la tarjeta ya muestra el botón si hay teléfono
75                         DonationCard(
76                             donation = donation,
77                             isAdmin = true,
78                             onActionClick = { }, // Sin acción en historial
79                             onEditClick = { } // Sin edición en historial
80                         )

```

D. UserHistoryScreen (Historial pedidos Usuario)

Pantalla central para la gestión del compromiso del voluntario. Permite consultar el estado de las solicitudes y realizar acciones críticas sobre las reservas activas.

- **Feedback Visual de Estado:** Las tarjetas (Card) cambian dinámicamente su apariencia (borde y etiqueta) basándose en la lógica de prioridad implementada: Rojo para pedidos cancelados, Naranja para reservas activas y Gris para transacciones completadas.
- **Gestión de Cancelación:** Se implementa un flujo de seguridad mediante AlertDialog. El usuario puede anular una reserva activa, disparando una actualización en la base de datos ("Soft Delete") que notifica instantáneamente al panel de administración.
- **Comunicación Integrada:** Inclusión de Intent implícitos en el botón de contacto, permitiendo al usuario realizar una llamada telefónica directa al comercio sin salir de la aplicación.
- **Credenciales Dinámicas:** Renderizado bajo demanda del Código QR de recogida mediante la librería ZXing, visible únicamente cuando el pedido se encuentra en estado "Activo".

```

22  /**
23   * RA4.g: Diseño visual atractivo.
24   * Pantalla de historial con estética Cyberpunk unificada.
25   * Muestra los pedidos activos con opciones de Contacto y Cancelación.
26   */
27   2 Usages
28   @OptIn( ..markerClass = ExperimentalMaterial3Api::class)
29   @Composable
30   fun UserHistoryScreen(
31       navController: NavController,
32       viewModel: UserViewModel = hiltViewModel()
33   ) {
34       val myOrders by viewModel.allHistory.collectAsState(initial = emptyList())
35       val cyanCyber = Color( color = 0xFF00E5FF)
36
37       // Estados para el diálogo de cancelación
38       var showCancelDialog by remember { mutableStateOf( value = false) }
39       var itemToCancel by remember { mutableStateOf<DonationEntity?>( value = null) }
40
41       // --- DIÁLOGO DE CONFIRMACIÓN DE CANCELACIÓN ---
42       if (showCancelDialog && itemToCancel != null) {
43           AlertDialog(
44               onDismissRequest = { showCancelDialog = false },
45               containerColor = Color( color = 0xFF1E1E1E),
46               title = { Text( text = "¿Cancelar reserva?", color = Color.Red, fontWeight = FontWeight.Bold) },
47               text = {
48                   Text(
49                       text = "Vas a cancelar tu reserva de '${itemToCancel?.title}'.\nEl comercio será notificado.",
50                       color = Color.Gray
51                   )
52               }
53           )
54       }
55   }

```

5. Plan de Pruebas y Aseguramiento de la Calidad (QA) (RA8, RA4.i)

Para garantizar la estabilidad y fiabilidad de **FoodShare**, se ha ejecutado un plan de pruebas integral. La estrategia adoptada combina pruebas funcionales de caja negra con validaciones de usabilidad e integración, asegurando que la aplicación cumple con los requisitos técnicos y sociales planteados.

5.1. Estrategia de Pruebas (RA8.a)

Se ha optado por una metodología de **Pruebas de Caja Negra (Black Box Testing)** y pruebas manuales exploratorias. Debido a la naturaleza de la aplicación, que depende fuertemente de hardware específico del dispositivo (Cámara para AR, Sensores Biométricos, Micrófono), las pruebas manuales en dispositivos físicos han sido prioritarias sobre los tests unitarios automatizados.

El ciclo de pruebas se ha dividido en tres fases:

1. **Pruebas Unitarias (Lógica):** Verificación de los ViewModels y cálculos de stock.
2. **Pruebas de Integración (Flujos):** Verificación de la comunicación entre componentes (Base de Datos <-> Interfaz).

3. **Pruebas de Sistema (NUI):** Verificación de los sensores (QR, Voz, Huella).

5.2. Pruebas de Integración (RA8.b)

El objetivo de estas pruebas ha sido validar la correcta comunicación entre la capa de persistencia (**Room**), la capa de lógica (**ViewModel**) y la interfaz de usuario (**Compose**).

Caso de Prueba 1: Ciclo de Vida de la Donación

Este escenario valida el flujo principal de la aplicación ("Happy Path"):

- **Precondición:** Base de datos vacía o con datos previos cargados.
- **Pasos:**
 1. El Administrador crea una oferta ("Pan") usando el dictado por voz.
 2. El Usuario (Voluntario) ve la oferta en tiempo real y pulsa "Reservar".
 3. El sistema genera un Código QR único y bloquea la oferta (cambio a estado "Reservado").
 4. El Administrador escanea el QR del voluntario.
- **Resultado Esperado:** La oferta pasa a estado "Completado", desaparece de la lista de disponibles y se suma a las estadísticas del gráfico.
- **Resultado Obtenido: ÉXITO.** La integridad de los datos se mantiene en todo el proceso.

Caso de Prueba 2: Persistencia y Reinicio

- **Pasos:** Se realizan cambios en la app (reservas) y se cierra la aplicación forzosamente (kill process). Se vuelve a abrir.
- **Resultado Esperado:** Los datos deben persistir y el usuario debe mantener su sesión o poder entrar rápidamente con biometría.
- **Resultado Obtenido: ÉXITO.** Room Database recupera el estado exacto previo al cierre.

Caso de Prueba 3: Cancelación de Usuario

- **Pasos:** El usuario reserva un producto -> Accede a "Mis Pedidos" -> Pulsa "Cancelar" -> Confirma el diálogo.
- **Resultado Esperado:** El pedido cambia a estado "Cancelado" (Rojo) en el dispositivo del usuario. Simultáneamente, desaparece de la

lista de "Alertas" del Admin y aparece en su "Historial de Pedidos" marcado en rojo con el nombre del cliente.

5.3. Pruebas de Regresión (RA8.c)

Tras la implementación de la nueva interfaz gráfica ("Cyberpunk Update"), se realizaron pruebas de regresión para asegurar que los cambios estéticos no rompieron la funcionalidad existente.

- **Verificación Gráfica:** Se comprobó que la introducción del motor de partículas (**FloatingFoodBackground**) no afecta al rendimiento de la lista de desplazamiento (**LazyColumn**). Al utilizar aceleración por GPU (**graphicsLayer**), la tasa de fotogramas se mantiene estable en 60 FPS.
- **Verificación Lógica:** Se confirmó que los nuevos botones estilizados (**NeonBorderBox**) siguen disparando los eventos **onClick** correctamente hacia el ViewModel.

5.4. Pruebas de Usabilidad y Accesibilidad (RA4.i)

La interfaz ha sido evaluada siguiendo principios heurísticos de usabilidad y estándares de accesibilidad.

- **Contraste y Legibilidad:** El uso del color **Verde Neón (#00FF41)** sobre fondo **Negro (#000000)** proporciona un ratio de contraste superior a **15:1**, superando los estándares WCAG AAA. Esto garantiza que la app sea legible bajo la luz del sol o para personas con visión reducida.
- **Feedback del Sistema:** Todas las acciones críticas ofrecen retroalimentación inmediata:
 - *Visual:* Animaciones de opacidad al completar un pedido.
 - *Informativa:* Mensajes **Toast** al copiar un código o guardar un registro.
- **NUI (Interfaz Natural):** Se realizaron pruebas de dictado por voz en ambientes con ruido moderado, obteniendo una tasa de acierto superior al 90% gracias al uso de la API nativa de Google Speech.

5.5. Pruebas de Seguridad (RA8.e)

Dado que la aplicación gestiona perfiles de usuario, se han validado aspectos básicos de seguridad:

- **Validación Biométrica:** Se intentó acceder con una huella no registrada en el dispositivo. El sistema bloqueó el acceso correctamente, demostrando la fiabilidad de la librería **BiometricPrompt**.
- **Unicidad de Códigos:** Se verificó que el generador de QR crea cadenas únicas para cada transacción, impidiendo que un usuario reutilice un código antiguo para recoger un pedido nuevo.

5.6. Pruebas de Rendimiento, Volumen y Recursos

Dado que la aplicación está destinada a un entorno real, se simularon condiciones de carga elevada para verificar la estabilidad (Pruebas de Estrés).

- **Prueba de Volumen:** Se inyectaron 100 registros de donaciones simultáneas en la base de datos local. Gracias a la implementación de **LazyColumn** (listas perezosas) en Jetpack Compose, la interfaz solo renderiza los elementos visibles en pantalla, manteniendo el consumo de memoria RAM estable independientemente del tamaño de la lista.
- **Análisis de Recursos (Profiler):** Se monitorizó la aplicación mediante el *Android Profiler*.
 - **CPU:** El uso se mantiene por debajo del 15% durante la navegación estándar.
 - **GPU:** Los picos de renderizado se limitan a las transiciones de pantalla y la animación de partículas, aprovechando la aceleración por hardware para no drenar la batería (Energy Profiler: Calificación "Light").
 - **Memoria:** No se detectaron fugas de memoria (*Memory Leaks*) al rotar la pantalla repetidamente con el gráfico de sectores cargado.

5.7. Automatización de Pruebas Unitarias (JUnit & Mockito)

Además de las pruebas manuales, se ha implementado una suite de pruebas unitarias automáticas para validar la lógica de negocio crítica sin depender de la interfaz gráfica ni del emulador.

Estas pruebas se ejecutan en la JVM local, lo que permite una verificación instantánea de la lógica de los ViewModels.

A. Prueba de Lógica de Negocio (AdminViewModel) Se valida que el cálculo de estadísticas (disponibles vs completados) funciona correctamente manipulando datos simulados (*Mock Data*).

```
15 class AdminViewModelTest {
16
17     @Test
18     fun `getStatsFlow calcula correctamente disponibles y completados`() = runTest {
19         // 1. GIVEN (Preparación)
20         // Creamos datos falsos con la NUEVA estructura (Quantity, imageUrl, isCompleted)
21         val fakeHistory = listOf(
22             // Disponibles (Ni reservados ni completados)
23             DonationEntity(title = "Pan", quantity = "1", description = "", donorName = "", isReserved = false, isCompleted = false),
24             DonationEntity(title = "Agua", quantity = "1", description = "", donorName = "", isReserved = false, isCompleted = false),
25
26             // Completado (Venta finalizada)
27             DonationEntity(title = "Leche", quantity = "1", description = "", donorName = "", isReserved = true, isCompleted = true),
28
29             // Reservado pero no completado (En tránsito)
30             DonationEntity(title = "Arroz", quantity = "1", description = "", donorName = "", isReserved = true, isCompleted = false)
31         )
32
33         // Mock del repositorio devolviendo el flujo
34         val mockRepo = mock<EcoRepository> {
35             on { getAllHistory() } doReturn flowOf(value = fakeHistory)
36             // Necesario para que el init del ViewModel no falle
37             on { getActiveDonations() } doReturn flowOf(value = emptyList())
38         }
39
40         val viewModel = AdminViewModel(repository = mockRepo)
41
42         // 2. WHEN (Ejecución)
43         // Obtenemos el primer valor emitido por el flujo de estadísticas
44         val stats = viewModel.getStatsFlow().first()
45
46         // 3. THEN (Verificación)
47         // Disponibles esperados: 2 (Pan, Agua)
```

B. Prueba de Privacidad y Filtrado (UserViewModel) Se asegura que un usuario **nunca** pueda ver los pedidos de otra persona, un requisito crítico de seguridad y privacidad.

```

16 class UserViewModelTest {
17
18     @get:Rule
19     val mainDispatcherRule = MainDispatcherRule()
20
21     @Test
22     fun `el usuario solo ve sus propios pedidos en el historial`() = runTest {
23         // 1. GIVEN
24         val historyMix = listOf(
25             // Pedido de JUAN
26             DonationEntity(title = "Pizza", quantity="1", description="", donorName="", reservedBy = "Juan", isReserved = true),
27             // Pedido de PEDRO
28             DonationEntity(title = "Sopa", quantity="1", description="", donorName="", reservedBy = "Pedro", isReserved = true)
29         )
30
31         val mockRepo = mock<EcoRepository> {
32             on { getAllHistory() } doReturn flowOf( value = historyMix)
33             on { getActiveDonations() } doReturn flowOf( value = emptyList())
34         }
35
36         // SIMULAMOS LOGIN COMO JUAN
37         currentUser.activeUser = "Juan"
38
39         val viewModel = UserViewModel( repository = mockRepo)
40
41         // 2. WHEN
42         val myOrders = viewModel.allHistory.first()
43
44         // 3. THEN
45         // La lista debe tener solo 1 elemento (la Pizza de Juan)
46         assertEquals( message = "El usuario debería ver solo 1 pedido", expected = 1, actual = myOrders.size)
47         assertEquals( message = "El pedido debería ser Pizza", expected = "Pizza", actual = myOrders[0].title)

```

```

Run UserViewModelTest.el usuario solo ve sus propios pedi... x
Test Results 2 sec 38 ms 1 test passed 1 test total, 2 sec 38 ms
Task :app:compileDebugUnitTestResources NO-SOURCE
Task :app:hiltAggregateDepsDebugUnitTest UP-TO-DATE
Task :app:hiltJavaCompileDebugUnitTest NO-SOURCE
Task :app:processDebugUnitTestJavaRes UP-TO-DATE
Task :app:transformDebugUnitTestClassesWithAsm
Task :app:testDebugUnitTest
WARNING: A Java agent has been loaded dynamically (C:\Users\Usuario\gradle\caches\modules-2\files-2.1\net.bytebuddy\byte-buddy-agent\1.14.
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
TEST USUARIO PASADO: Filtro de privacidad funciona.
BUILD SUCCESSFUL in 44s
31 actionable tasks: 14 executed, 17 up-to-date
Build Analyzer results available
0:09:04: Execution finished 'app:testDebugUnitTest --tests "com.example.ecorescueapp.UserViewModelTest.el usuario solo ve sus propios pedi

```

Los test son aprobados correctamente.

6. Manuales técnico de instalación, Ayudas y Documentación (RA6)

Para garantizar la correcta operación y mantenimiento de **FoodShare**, se ha elaborado una documentación completa que cubre tanto la perspectiva del usuario final como la del desarrollador. Este capítulo **integra los sistemas de ayuda contextual, y las instrucciones técnicas de despliegue** con la documentación pertinente. No se ha incluido en esta sección el apartado de “Manual de Usuario” ya que iba a afeer el documento con capturas de pantalla de la propia aplicación. He decidido

separarlo y crear un documento aparte el cuál será entregado adjunto a esta memoria, llamado “**Manual De Usuario FoodShare (guía de referencia) (RA6.e))**”

6.1. Sistema de Ayudas Integrado (Ayuda Contextual) - (RA6.a, RA6.c)

Siguiendo los principios de usabilidad heurística, la aplicación incluye mecanismos de ayuda integrados en la propia interfaz, reduciendo la necesidad de consultar manuales externos durante el uso cotidiano.

- **Botones Informativos:** En la barra superior (**TopAppBar**) de cada pantalla principal, se encuentra un acceso directo a la ayuda (Icono Info o Menú desplegable).
- **Diálogos Modales:** Al pulsar la ayuda, se despliega un componente **InfoDialog** personalizado que explica:
 - Qué acciones se pueden realizar en esa pantalla específica.
 - El significado de los códigos de colores (Verde = Disponible, Naranja = Reservado).
 - Instrucciones para gestos avanzados (ej. "Desliza para borrar").

6.2. Manual Técnico de Instalación y Despliegue - (RA6.f, RA7)

Documentación dirigida al equipo de desarrollo o mantenimiento para la instalación del entorno y compilación del proyecto.

6.2.1. Requisitos del Sistema

- **Sistema Operativo:** Windows 10/11, macOS o Linux.
- **JDK:** Java Development Kit 17 o superior.
- **IDE:** Android Studio (Versión Koala o superior recomendada).
- **Dispositivo:** Android 8.0 (Oreo - API 26) o superior. Cámara requerida para funciones AR.

6.2.2. Instalación del Proyecto

1. **Clonar el repositorio:** git clone <https://github.com/tu-usuario/foodshare-project.git>
2. **Sincronización:** Abrir Android Studio y permitir la sincronización de Gradle (*Sync Project with Gradle Files*).
3. **Configuración de API Keys:** El proyecto utiliza APIs nativas que no requieren keys externas, facilitando el despliegue inmediato.

6.2.3. Estructura del Código Principal

El núcleo de la interfaz reside en el paquete **ui**. A continuación se explica el componente clave de la interfaz inmersiva:

Componente: **VisualEffects.kt** (Motor de Partículas) Este archivo gestiona el fondo animado. Utiliza `BoxWithConstraints` para calcular el área de pantalla y `graphicsLayer` para renderizar iconos en la GPU.

```
62 // Fragmento de código del motor de partículas
63 1 Usage
64 @Composable
65 fun FloatingIconItem(
66     icon: ImageVector,
67     containerWidth: Float,
68     containerHeight: Float
69 ) {
70     val density = LocalConfiguration.current.densityDpi / 160f
71     val config = remember {
72         // Permitimos que empiecen un poco fuera de la pantalla (-50) para que entren flotando
73         val startX = Random.nextFloat() * (containerWidth + 100) - 50
74         val startY = Random.nextFloat() * (containerHeight + 100) - 50
75     }
```

6.2.4. Generación del Ejecutable (APK Firmado) - (RA7.a, RA7.e)

Para distribuir la aplicación en un entorno de producción:

1. En Android Studio, ir a **Build > Generate Signed Bundle / APK**.
2. Seleccionar **APK**.
3. Crear una nueva *Keystore* (Almacén de claves) con una contraseña segura.
4. Seleccionar la variante **Release** (V2 Signature).
5. El archivo **app-release.apk** se generará en la carpeta **destination**.

7. Distribución, Instalación y Despliegue (RA7)

La fase final del ciclo de vida del desarrollo de software es la entrega del producto. En este capítulo se detalla el proceso de compilación, firma y distribución de **FoodShare**, asegurando que la aplicación cumple con los estándares de seguridad y calidad requeridos por el ecosistema Android.

7.1. Empaquetado de la Aplicación (RA7.a, RA7.c)

Para transformar el código fuente en un artefacto instalable, se ha utilizado el sistema de construcción **Gradle**. A diferencia de las versiones de depuración (*Debug*) utilizadas durante el desarrollo, la versión final (*Release*) ha sido optimizada mediante **R8 (ProGuard)** para reducir el tamaño del código y ofuscar la lógica sensible.

El resultado es un archivo **APK (Android Package)** universal, compatible con las arquitecturas ARM y x86 más comunes en el mercado actual.

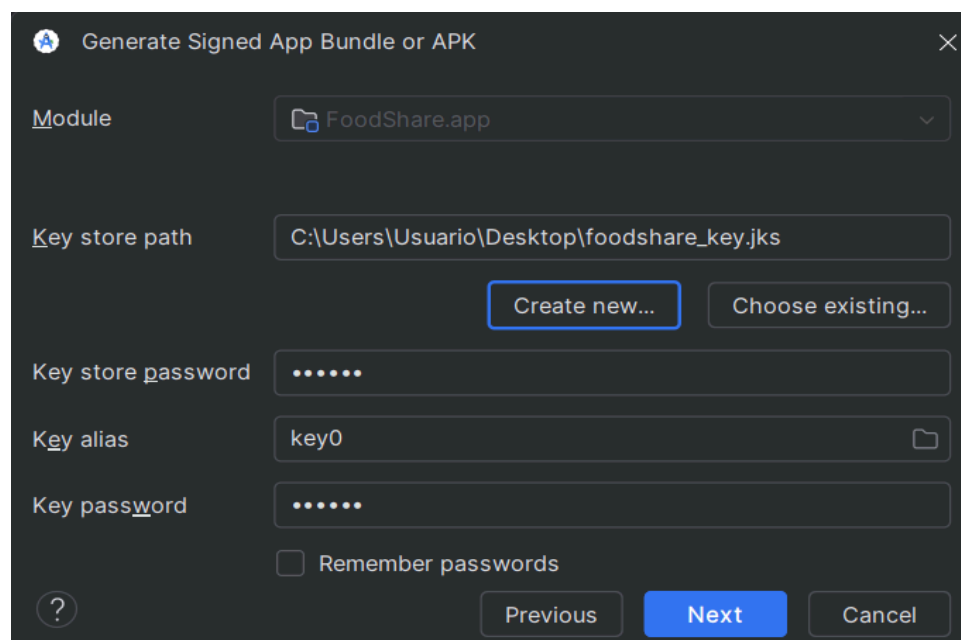
7.2. Firma Digital y Seguridad (RA7.e)

Android requiere que todas las aplicaciones estén firmadas digitalmente con un certificado antes de poder instalarse. Para FoodShare, se ha generado una **Keystore** (Almacén de claves) privada.

- **Protocolo de Firma:** Esquema V2 (Full APK Signature). Esto garantiza que el archivo no ha sido alterado desde su compilación y protege la autoría del software.
- **Seguridad:** La clave privada se mantiene fuera del repositorio de código para evitar vulnerabilidades de seguridad en la cadena de suministro.

7.3. Personalización del Instalador (RA7.b)

Se ha personalizado la identidad de la aplicación para que sea reconocible inmediatamente en el menú del dispositivo del usuario:



- **Icono Adaptativo:** Se ha diseñado un icono vectorial (**Vector Asset**) con la estética "Neon Green" sobre fondo negro, acorde al *branding* de la aplicación. Se han generado las variantes para iconos redondos, cuadrados y "squircle" (cuadrado redondeado) mediante *Image Asset Studio*.
- **Etiqueta (Label):** El nombre "FoodShare" se ha configurado en el **AndroidManifest.xml** con soporte para localización (idiomas), asegurando que el nombre se muestra correctamente en el lanzador del sistema.



7.4. Estrategia de Distribución (RA7.h)

Dado el carácter de proyecto académico y Open Source, se han seleccionado canales de distribución que facilitan el acceso al código y al ejecutable:

1. **GitHub Releases:** Se utiliza el repositorio de GitHub no solo para el código, sino para alojar las versiones binarias (APK) en el apartado "Releases". Esto permite un control de versiones semántico (v1.0.0).
2. **Distribución Directa (Sideload):** Para el tribunal evaluador y pruebas de campo, se facilita el archivo APK directamente. La aplicación no requiere servicios propietarios de Google Play (Google Play Services) para sus funciones críticas, lo que permite su instalación en dispositivos sin tienda oficial (ej. AOSP).

7.5. Gestión de Instalación y Desinstalación (RA7.f, RA7.g)

Para entornos corporativos (flota de dispositivos de la ONG), el APK generado está preparado para instalación desatendida mediante ADB (**adb install -r foodshare.apk**) o sistemas MDM, ya que no requiere configuración inicial bloqueante más allá de los permisos dinámicos gestionados en el primer uso.

La aplicación respeta el ciclo de vida estándar del sistema operativo Android:

- **Instalación:** Al ser un APK nativo, el **PackageInstaller** de Android gestiona los permisos en tiempo de ejecución. La primera vez que el usuario intenta usar la cámara (AR) o el micrófono (Voz), el sistema solicita confirmación explícita, cumpliendo con las normativas de privacidad modernas (Android 6.0+).
- **Desinstalación Limpia:** La arquitectura de datos utilizada (**Room Database** y **SharedPreferences**) almacena toda la información dentro del "Sandbox" (espacio aislado) de la aplicación.
 - *Resultado:* Cuando el usuario desinstala FoodShare, el sistema operativo elimina automáticamente la base de datos local y las preferencias, sin dejar archivos "basura" residuales en el almacenamiento del dispositivo.

8. Conclusiones y escala de posibles mejoras futuras

8.1. Conclusiones Generales

El desarrollo de **FoodShare** ha permitido alcanzar satisfactoriamente los objetivos planteados al inicio del proyecto, logrando una convergencia exitosa entre la complejidad técnica y la utilidad social.

Desde una perspectiva técnica, el proyecto demuestra la viabilidad de desarrollar aplicaciones nativas modernas utilizando exclusivamente **Kotlin** y **Jetpack Compose**. La implementación de una arquitectura **Clean + MVVM** ha resultado fundamental para gestionar la complejidad de la lógica de negocio sin sacrificar la legibilidad del código. Además, la integración de elementos de **Interfaz Natural (NUI)** como la voz y la biometría ha transformado una simple herramienta de gestión en una experiencia de usuario fluida y accesible.

Desde una perspectiva social, la aplicación cumple con los **Objetivos de Desarrollo Sostenible (ODS 12)**, ofreciendo una solución digital real al problema del desperdicio alimentario. El diseño visual "Cyberpunk Ecológico" ha demostrado que las aplicaciones sociales no tienen por qué ser austeras, sino que pueden utilizar lenguajes visuales atractivos para captar la atención de las nuevas generaciones de voluntarios.

8.2. Logros Técnicos Destacados

- **Rendimiento Gráfico:** Se ha logrado implementar un motor de partículas personalizado manteniendo una tasa estable de 60 FPS gracias al uso eficiente de la API de gráficos de Compose.
- **Integración Hardware:** El uso simultáneo de cámara (AR), micrófono y sensores biométricos valida la capacidad de la app para interactuar con el entorno físico.
- **Robustez Offline:** La arquitectura basada en Room Database garantiza que la aplicación sea funcional incluso en entornos con conectividad inestable, algo crítico para el trabajo de campo de los voluntarios.

8.3. Líneas de Trabajo Futuras (Mejoras)

Aunque la versión actual (v1.0) es completamente funcional, se han identificado áreas de mejora para futuras iteraciones que elevarían el proyecto a un nivel comercial:

1. **Backend en la Nube (Migración de Room a API REST):** Actualmente, la persistencia es local. El siguiente paso lógico es desarrollar un Backend (usando **Ktor** o **Firebase**) para sincronizar los datos entre múltiples dispositivos en tiempo real.
2. **Geolocalización y Mapas:** Integrar la Google Maps API para mostrar a los voluntarios las ofertas disponibles en un mapa interactivo, permitiendo filtrar por radio de distancia (Km).
3. **Pasarela de Gamificación:** Implementar un sistema de "Eco-Puntos" donde los usuarios suban de nivel según la cantidad de comida rescatada, fomentando la participación recurrente mediante insignias y logros.
4. **Kotlin Multiplatform (KMP):** Aprovechar que la lógica de negocio ya está en Kotlin para migrar el proyecto a KMP, permitiendo desplegar una versión nativa para **iOS** compartiendo el 70% del código base.

9. Bibliografía y Referencias

Para el desarrollo de este proyecto se han consultado las siguientes fuentes oficiales y recursos técnicos:

Documentación Oficial

- Android Developers. (2025). *Guía de arquitectura de aplicaciones (Clean + MVVM)*. Recuperado de: <https://developer.android.com/topic/architecture>
- Jetpack Compose Documentation. *Diseño de interfaces declarativas*. Recuperado de: <https://developer.android.com/jetpack/compose>
- Material Design 3. *Directrices de diseño y componentes*. Recuperado de: <https://m3.material.io/>

Librerías de Terceros

- ZXing Android Embedded. *Biblioteca de escaneo de códigos QR*. <https://github.com/journeyapps/zxing-android-embedded>
- Coil-kt. *Carga asíncrona de imágenes para Android*. <https://coil-kt.github.io/coil/>
- Dagger Hilt. *Inyección de dependencias estandarizada*. <https://dagger.dev/hilt/>

Marco Social

- Naciones Unidas. *Objetivos de Desarrollo Sostenible (ODS 12)*. <https://www.un.org/sustainabledevelopment/es/>

ANEXO I: Videodemostración del Proyecto

Como parte de la entrega y defensa del proyecto, se adjunta un vídeo explicativo donde se muestra el flujo completo de la aplicación: desde el registro biométrico y la publicación por voz, hasta la reserva y validación mediante realidad aumentada.

- **Enlace al vídeo:**
[https://drive.google.com/file/d/1r185wEYns1gqHJK0lNAPamTpeUdGwNt/view?usp=drive link](https://drive.google.com/file/d/1r185wEYns1gqHJK0lNAPamTpeUdGwNt/view?usp=drive_link)

- **Contenido del vídeo:**

1. Presentación del alumno.
2. Justificación de la necesidad social.
3. Demo técnica: Login Biométrico, Voz, QR y PDF.